Classes Part-4

Topics are →
- Singleton class
- Immutable class
- Wrapper class

Singleton class

→ A Singleton class is a class that allows only one object (instance) to be created during the lifetime of an application and provides a global point of access to that object.

Ex: Think of it like a printer manager in an office. There should be only one manager coordinating all print jobs, not multiple managers.

Characteristics:

1. Only one instance of the class exists
2. The constructor is private, so no one can create objects using new.
3. The class provides a public static method to access the single instance.

Different ways of creating Singleton class:

- Eager Initialization
- lazy Initialization
- Synchronization Block
- Double check lock (there is a memory issue, resolved through volatile inst variable)
- Bill pugh solution
- Enum Singleton.

1. Eager Initialization:
Disadvantage: even if conObject is not used but still it is initialized egerly (in advance)
Ex:

```
public class DBConnection{
    private static DBConnection conObject = new DBConnection();
    private DBConnection(){ }

    public static DBConnection getInstance(){
        return conObject;
    }
}
public class Main{
    psvm(){
        DBConnection connObject = DBConnection.getInstance();
    }
}
```

Eager Initialization →

2. Lazy Initialization:
Ex:

conobj is not initialized at load time ↓

```
public class DBConnection{
    private static DBConnection conObject;
    private DBConnection(){ }
    public static DBConnection getInstance(){
        if(conObject == null){
            conObject = new DBConnection();
        }
        return conObject;
    }
}
```

Page No :-
Date :- / /

3. Synchronized Method :

→ Ex :

```
public class DBConnection{
    private static DBConnection conObject;
    private DBConnection(){}

    Synchronized public static
            DBConnection getInstance(){
        if(conObject == null){
            conObject = new DBConnection();
        }
        return conObject();
    }
}
```

Synchronized keyword is used to acquire Lock on getInstance() method so only one process/thread used it at a time, after finising it unlocks it so other thread use it.

Place1 call method → Lock/unlock
Place2 " ______
:
Place n " ______

} synchronized used at method level so, it is a time consuming process that slow down the program.

4. Double check locking
→ it adds concurrency

```
public class DatabaseConnection {
    private static volatile DatabaseConnection conObj;
    private DatabaseConnection() { }
    public static DatabaseConnection getInstance() {
        if (conObj == null) {          // check 1
            Synchronized (DatabaseConnection.class) {
                if (conObj == null) {  // check 2
                    conObj = new DatabaseConnection();
                }
            }
        }
    }
```

use → any read/write open happen to memory not in cache

Disadvantage: Since method uses volatile keyword so all read/write operation happens through memory not through the cache so we can say it still a slow process. also Synchronized used that add extra overhead due to lock/unlock.

5. Bill pugh Solution:

→ It also uses eagen initialization of object but wrap in another nested class. so when class is loded inner class doesn't initialized at the same time it will be initialized only when it will refer nested class then it will be loaded.

Ex:

```
public class DBConnection{
    private DBConnection(){ }

    private static class DBConnectionHelper{
        private static final DBConnection
            INSTANCE_OBJ = new DBConnection();
    }

    public static DBConnection getInstance(){
        return DBConnectionHelper.INSTANCE_OBJ;
    }
}
```

6. ENUM Singleton

→ By default ENUM constructor are private and in each JVM only one snapshot of ENUM variable is present so bydefault ENUM are singleton also.

```
enum DBConnection {
    INSTANCE;
}
```

Immutable Class:

- We can not change the value of an object once it is created.
- Declare class as 'final' so that it can not be extended.
- All class members should be private. so that direct access can be avoided
- And class members are initialized only once using constructor.
- There should not be any setter methods, which is generally use to change the value.
- Just getter methods. And returns copy of member variables.
- Ex: String, wrapper classes etc.

⇒
```
final class MyImmutableClass {
    private final String name;
    private final List<Object> petNameList;

    MyImmutableClass(String name, List<Object>
                                    petNameList) {
        this.name = name;
        this.petNameList = petNameList;
    }
    public String getName() {
        return name;
    }
```

```
public List<Object> getPetNamelist() {
    // this is req, because making list final
    // means you can now point it to new list,
    // but still can add, delete values in list.
    // So that's why we send the copy of it.

    return new Arraylist<>(petNamelist);
  }
}
public class Main {
  psum() {
    List<Object> petNames = new Arraylist<>();
    petNames.add("cow");
    petNames.add("cat");

    MyImmutableClass obj = new
          MyImmutableClass("Mohit", petNames);
    obj.getPetNamelist().add("dog");
    Sout(obj.getPetNamelist());
  }
}
```

Output : ["cow", "cat", "dog"]